```
      applyBase = =>
        result = base.apply @, (params.concat callback)
      decoration.apply @, (params.concat applyBase)
      result

afterAsync = (decoration) ->
  (base) ->
    (params..., callback) ->
      decorated = (params...) =>
        decoration.apply @, (params.concat -> (callback.apply @,
params))
      base.apply @, (params.concat decorated)
```

There are other ways to deal with resources that are accessed asynchronously, such as treating them as streams of data, as you'll learn about in chapter 9.

Continuations

This concept of passing in the rest of your program as a callback is similar to something called *continuation-passing style*. One of the criticisms of CoffeeScript in its current state is that callbacks all have to be nested. The asynchronous composition techniques presented here and the techniques shown in chapter 9 go a long way to helping solve that problem. Further, in chapter 13 you'll see how JavaScript, and hence CoffeeScript, is evolving to have better language techniques for dealing with asynchronous code.

Later on, over coffee (robots drink coffee too), Agtron tells you what his philosophy professor once told him, "Our understanding of the world often gets tangled up until eventually it becomes one big knot. Philosophy is about teasing out those knots and untangling things so that you have simple explanations of things." The same thing applies to the world of a program. Sometimes things get tangled up. If you pull harder on the knot, you make it worse. If you do surgery on the knot by cutting it, you can break the entire universe. The point is to instead tease out the tangles and make your program less intertwined. The techniques you've seen can help you to tease out some of the knots in your program.